Grid dimensioning and data formatting
Advanced computer architecture

(B-KUL-JLI1QS)

March 21, 2024



Session Date:	March 15, 2024
Partners:	Emma Dejagere
	Brecht Van de Sijpe
Class:	MElsw
Instructor:	Alex Marinsek

1 Introduction

This labsession focusses on two subjects. The first part looks into the difference between coalesced and "uncoalesced" usage of the GPU memory. The second part looks into the differences between using global, shared and constant memory.

2 Method

To determine the difference between coalesced and uncoalesced usage of the memory we assign the threads to an arrayindex in a specific way. The same program as in labsession three is used and the graph generated in this session is recreated in both the coalesced and uncoalesced manner of assigning elements to threads. To obtain the uncoalesced way of assigning we assign based on threadId, blockId and blockDim. First the coalesced index is calculated based on these three parameters: $index = blockId.x * blockDim.x + threadId.x$. To make the memory access uncoalesced we make neigbouring threads access memory that is at the other side of the array. When $index\%2 = 0$ the index is calculated as the inverse index in the array. For example in an array of 10 elements the thread that originally gets index 2 will get the element at index 7, the thread at index 4 will get the element at index 5 and so on... Coalesced memory access combines threads into a single memory transaction, which should lead to a higher memory throughput.

To study the effect of shared, global and constant memory three kernels are implemented which each use the specific type of memory. Since constant memory is very limited only arraysizes up to 89 are allowed in our case. For this reason two graphs are generated: one to measure the difference between shared and global memory more precisely and one to measure the difference between constant and global / shared memory. Since the effects of shared and global memory should not be visible when using a small array size we cannot conclude anything about this in these last graphs.

3 Results

Looking at the first graph, it can be noted that, when comparing the coalesced results to the uncoalesced results, the coalesced memory access is more efficient than uncoalesced memory access on figure 1. Figure 2 doesn't confirm this and even contradicts this.

For shared vs global memory access we see that in both cases the calculating time increases when the length of the array increases. However, the calculating time of the global memory is slightly higher then that of the shared memory. This is explained because shared access provides faster accesses.

The calculating time increases when the array length increases for all three types of memory access. However, the calculating time of the constant memory is much higher then the shared and global memory access. We don't have an explanation as to why this is so much higher.

4 Conclusion

To conclude we have found that the performance of the shared memory is better than the performance of the global memory. The coalesced memory is more efficient then the uncoalesced

memory. There is a large difference in calculating time between constant memory and shared or global memory. Coalesced memory usage improves the performance depending on which GPU you use.

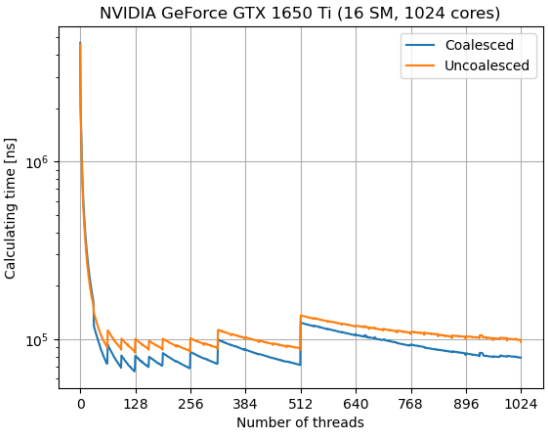


Figure 1: Coalesced vs uncoalesced on NVIDIA GeForce GTX 1650 Ti

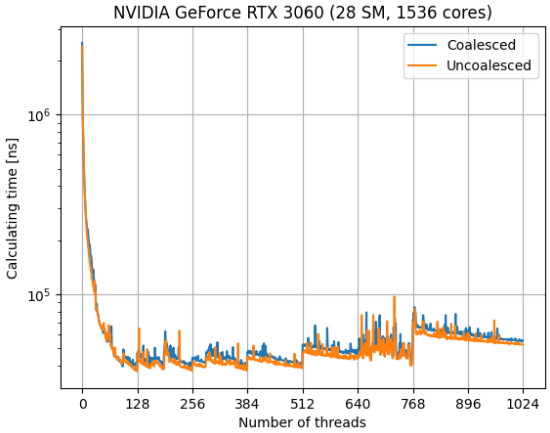


Figure 2: Coalesced vs uncoalsced on NVIDIA GeForce RTX 3060

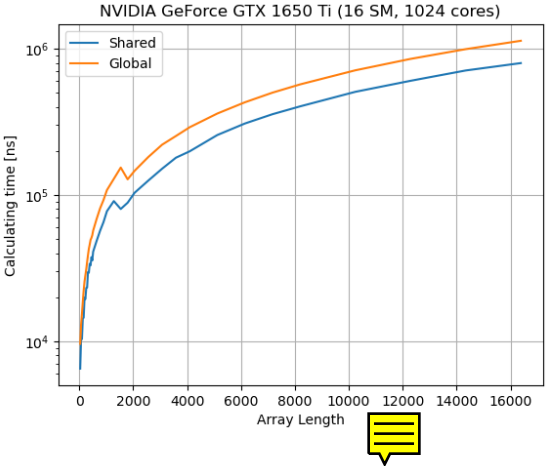


Figure 3: Shared vs global memory on NVIDIA GeForce GTX 1650 Ti

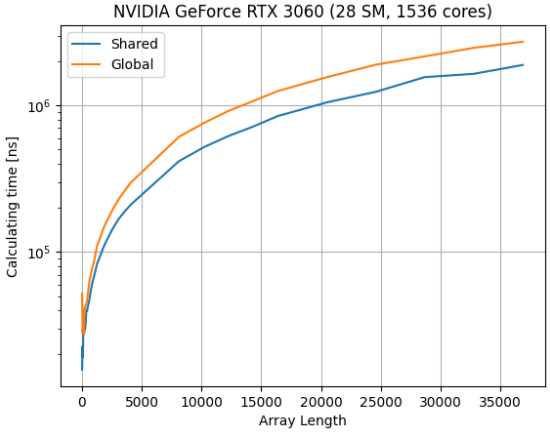


Figure 4: Shared vs global memory on NVIDIA GeForce RTX 3060

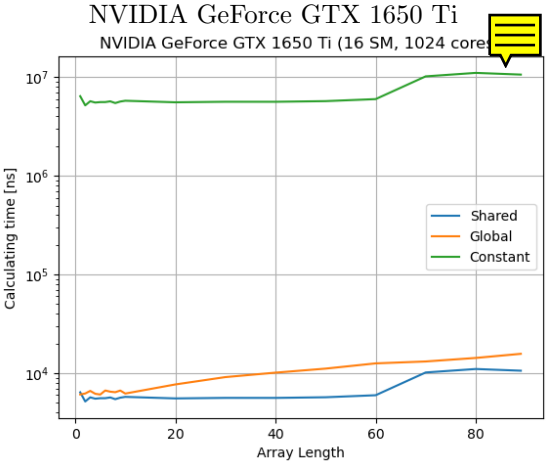


Figure 5: Shared / global vs constant on NVIDIA GeForce GTX 1650 Ti

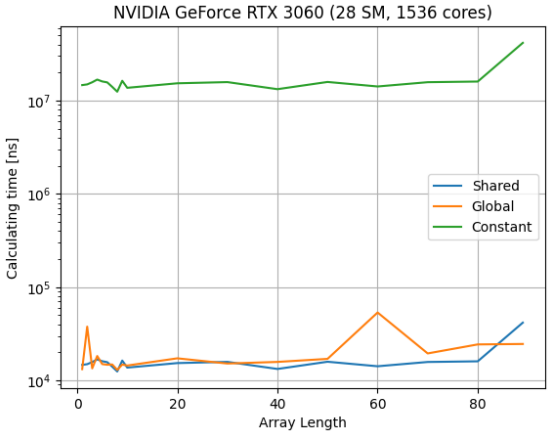


Figure 6: Shared / global vs constant on NVIDIA GeForce RTX 3060